

Cryptography & Network Security Notes

1. Introduction to Cryptography & Symmetric Encryption

- **Symmetric Encryption** also referred to as conventional or single-key encryption.
- **Cipher:** Is a method of transforming information so that it becomes unreadable to anyone who doesn't have the key to decode it.
- **Cipher = Normal text + Ciphertext**
- **Plain text:** Normal text / intelligible message / data fed into algorithm as input.
- **Ciphertext:** The process of plaintext $\rightarrow$ ciphertext & ciphertext $\rightarrow$ plaintext process is known as enciphering / encryption and deciphering / decryption.
- **Many scheme used for encryption constitute the area of study known as cryptography. Such schemes are known as cryptographic system on a cipher.**
- **Cryptanalysis:** Techniques used to decrypt the info without any knowledge of enciphering
Cryptology = Symmetric Cipher + Cryptanalysis
- **Symmetric Encryption model has five ingredients:**
 - **Plaintext:** Original intelligible message/data fed into algorithm as input.
 - **Encryption algorithm:** Performs various substitutions & transformations on the plaintext.
 - **Secret key:** It is an independent input value used alongside an encryption algorithm to change plaintext into unique ciphertext. Each substitution & transformations are entirely determined by the secret key.
 - **Ciphertext:** This is the scrambled message produced as output. Depends on plaintext & secret key. Even if we use two different keys to decrypt the same message twice, we will get complete different message. We can't easily decrypt the data with we have proper secret key to decrypt.
 - **Decryption algorithm:** This is exactly the encryption algorithm run in reverse. It takes secret key and ciphertext to produce original plain message.

Symmetric Encryption Model / Flowchart

Plain Text (X) $\rightarrow$ [Encryption Algorithm $E(K, X)$] $\rightarrow$ *Ciphertext (Y)* $\rightarrow$ [Decryption Algorithm $D(K, Y)$] $\rightarrow$ *Plain Text (X)*

Secret Key K is shared securely between sender and receiver.

Mathematical Representation:

- Plain text $X = [x_1, x_2, \dots, x_m]$

- Key $K = [k_1, k_2, \dots, k_j]$
- Ciphertext $Y = [y_1, y_2, \dots, y_n]$
- Encryption formula: $Y = E(K, X)$
- Decryption formula: $X = D(K, Y)$
- We don't need to keep algorithm secret, we need to keep only the key secret. This feature makes symmetric encryption feasible and widespread use.
- If they focus only on current messages (X), both current & future messages (K) success here, breaks the security of system for as long as that key is used.
- The opponent may see Y but not see X or have access to K or X .

2. Color Mixing Analogy & Encryption Concepts

The Color Mixing Trick (Analogy):

- In this trick they both chose same public color like black & blue, then both choose private color like yellow then mix it with yellow give light blue & dark blue.
- The hacker may see during the exchange and see what colour in real life it's not like mixing the colors in real life it's complete different using maths for it so it's difficult even if they try to change number the other end they don't receive proper message but also hacker do not.
- Known the message: Ciphertext: This is the scrambled message produced as output. Depends on plaintext & secret key. Even if we use two different keys to decrypt the same message twice, we will will get complete different message. We can't easily decrypt the data with we have proper secret key to decrypt.
- Yellow the hacker can interrupt here but both exchange color and add there secret colors like light blue + black & dark yellow blue resulting in navy blue where hackers cannot reverse engineer it.

Step-by-Step Color Mixing Table (Diffie-Hellman / Key Exchange Analogy)

Step	Alice	Bob	What a Hacker See / Notes
1. Public Base	Starts with public color: Yellow	Starts with same public color: Yellow	Hacker sees: Yellow
2. Private Choice	Chooses a private color: Black	Chooses a private color: Blue	Private colors are kept secret
3. The Mix	Mixes: Yellow + Black = Dark Yellow	Mixes: Yellow + Blue = Light Blue	Hacker sees Dark Yellow & Light Blue being exchanged
4. The Swap	Sends Dark Yellow to Bob	Sends Light Blue to Alice	Hacker intercepts the mixes, but cannot reverse engineer the private colors
5. Final Key	Takes Bob's mix & adds his own secret color: <i>Light blue + Black = Navy Blue</i>	Takes Alice's mix & add his own secret color: <i>Dark yellow + Blue = Navy Blue</i>	Engineers the final Navy Blue key

Note: Since the mixes are mathematical, it is impossible to un-mix.

3. Basic Cryptographic Transformations

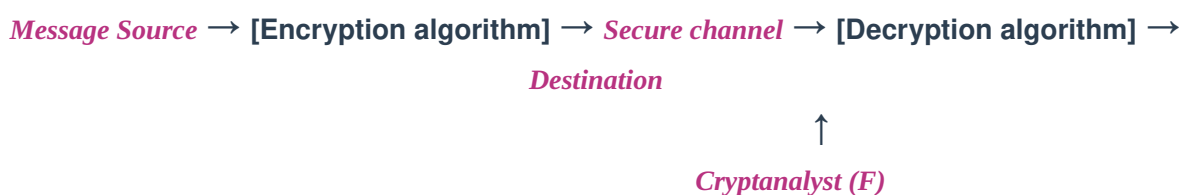
- Autography / Cryptography:

- 1. Substitution: The plaintext is mapped to another element like text in which position is same but value is different.
- 2. Transportation / Transposition: The elements are rearranged which is like write diagonally read horizontally. The value is same but position changes.

- Number of Keys Use:

- If one key is used to encrypt & decrypt the process is fast. But key sharing is difficult → Called symmetric encryption.
- If two keys are used like a public key to encrypt & private key to decrypt the process slow but key storage is protected → One time & stream cipher process bit by bit.

Cryptanalysis Attack Model Diagram



4. Hashing & Hash Functions

- **Hashing:** It is the process of converting an input of any size (like a word, a large file or a password) into a fixed-size string of characters, typically fixed-size string of characters, using mathematical formula called hashing function. The resulting output is called hash value, hash code or digest.
- We cannot convert digest again into the plain text.
- Every unique piece of data generates its own specific hash.
- **Avalanche Effect:** If you change even a single character in original data, the resulting hash will look completely different → a phenomenon known as avalanche effect.
- **Core Rules:**
 1. **Deterministic:** The exact same input will always produce the exact same output hash.
 2. **Fixed length o/p:** No matter if input is a single letter or an entire encyclopedia, the generated hash length remains exactly the same.
 3. **Irreversible (One-way):** You cannot reverse-engineer or decrypt a hash.
- **Practical Example (Password Storage):** The companies do not store plain-text passwords. Instead, they hash your password & store the hash. When you log in the hashes match. If hacker steal the database they get useless hashes not anything.
- **Algorithms Used:**
 - **Secure Algorithms:** SHA-256, SHA-3 and bcrypt. These are highly robust & widely used today for cryptography & passwords.
 - **Legacy / Broken Algorithms:** MD5 & SHA-1. Over time, computers became powerful enough to cause "hash collision". No longer considered safe for security.

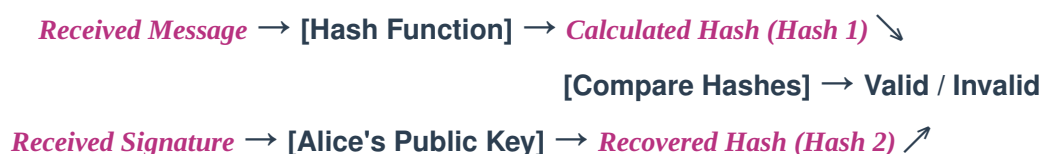
5. Digital Signatures

- **Digital Signatures:** A digital signature is a cryptographic technique used to authenticate the identity of the sender and ensure the integrity and non-repudiation of a digital message or document. It is created using the sender's private key and verified using the sender's public key.
- Who sent message/document → Authentication
- The message wasn't changed → Integrity
- The sender can't easily deny signing it → Non-repudiation
- Alice's Public Key: Can be shared with everyone. Used by Bob to verify Alice's Signature (* Private key → Sign, * Public key → Verify).
- Instead of signing the entire message directly, we first pass the message through a hash function.
ex: message → Hash Function → message Digest/Hash
- Bob can detect that the message was modified.

Process of Complete Digital Signature (Sender - Alice & Receiver - Bob)

1. Alice writes the message.
2. Message is passed through a hash function → A message digest/hash is generated.
3. Alice uses her private key to sign the hash → The resulting value is the digital signature.
4. Alice sends: *Message + Digital Signature*
5. Receiver - Bob:
 - Bob receives message & digital signature.
 - Bob calculates hash of received message.
 - Bob uses Alice's public key to verify/decrypt the digital signature → *Calculated hash vs Received signed hash.*
 - Bob compares 2 hashes.
 - If *Hash 1 == Hash 2* → Signature is valid. If mismatch → Signature invalid.

Digital Signature Verification Flowchart



Bob Key Summary:

- **Alice's Private Key: Kept secret by Alice, used to create/sign the digital signature. Nobody else should have access to it.**